Roteiro Prático: Sistema de Ordenha Bovina

Banco de Dados — PostgreSQL

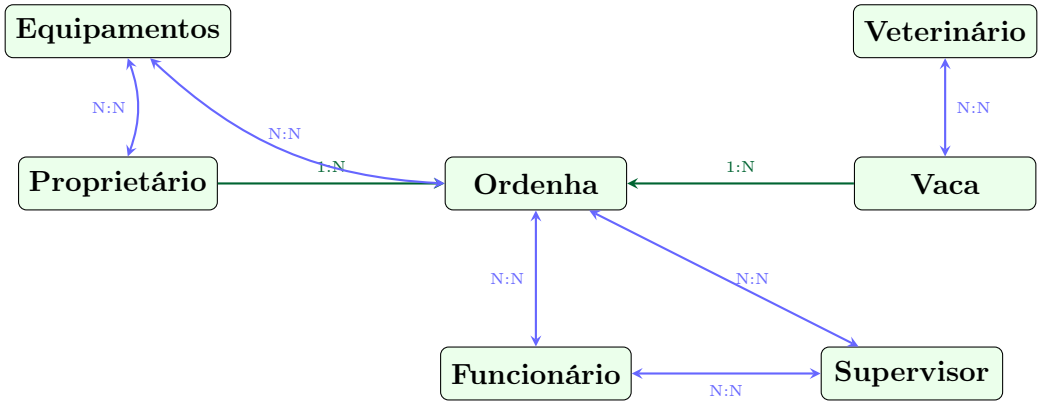
Tema: Modelagem Relacional, DDL e DML em Contexto Real

Objetivo da Aula Prática

Implementar passo a passo o banco de dados de um sistema de ordenha bovina, praticando: criação de tabelas, definição de chaves primárias e estrangeiras, relacionamentos 1:N e N:N, inserção de dados e consultas básicas. Ao final, o aluno deve ser capaz de executar o script completo no PostgreSQL e verificar a integridade referencial do modelo.

1 Visão Geral do Domínio

O sistema modela uma **fazenda produtora de leite**. As principais entidades e relacionamentos são descritos abaixo:



A tabela resumo das entidades e suas responsabilidades:

Entidade	Papel no sistema	Tipo de relação
Proprietário	Dono do animal/equipamento	1:N com Ordenha
Vaca	Animal ordenhado	1:N com Ordenha
Veterinário	Profissional de saúde animal	N:N com Vaca
Funcionário	Operador da ordenha	N:N com Ordenha
Supervisor	Responsável pelo turno	N:N com Ordenha e Funcionário
Equipamentos	Máquinas e instrumentos	N:N com Ordenha e Proprietário
Ordenha	Evento central do sistema	Tabela fato

2 Passo 1 — Limpar o Banco (DROP)

Atenção

Execute o bloco `DROP` **apenas** quando quiser recomeçar do zero. A cláusula `CASCADE` remove automaticamente todas as dependências. A opção `IF EXISTS` evita erros se as tabelas ainda não existirem.

```
DROP TABLE IF EXISTS
    Funcionario_Supervisor,
    Proprietario_Equipamento,
    Ordenha_Equipamento,
    Ordenha_Supervisor,
    Ordenha_Funcionario,
    Vaca_Veterinario,
    Ordenha,
    Vaca,
    Equipamentos,
    Veterinario,
    Funcionario,
    Supervisor,
    Proprietario
CASCADE;
```

Listing 1: Remoção segura de todas as tabelas

Dica Prática

A **ordem de remoção** importa: tabelas associativas (N:N) devem ser dropadas **antes** das tabelas que elas referenciam. Por isso listamos `Funcionario_Supervisor`, `Vaca_Veterinario`, etc. primeiro.

3 Passo 2 — Criar as Tabelas Principais (DDL)

3.1 Entidades independentes

As tabelas a seguir **não possuem chaves estrangeiras** e devem ser criadas primeiro, pois serão referenciadas pelas demais.

```
CREATE TABLE Proprietario (
    id                SERIAL PRIMARY KEY,
    nome_completo     VARCHAR(100),
    telefone          VARCHAR(20)
);

```

```sql
CREATE TABLE Equipamentos (
    codigo_equipamento SERIAL PRIMARY KEY,
    tipo               VARCHAR(50),
    ultima_manutencao  DATE,
    situacao           VARCHAR(50)
);

CREATE TABLE Vaca (
    id              SERIAL PRIMARY KEY,
    data_nascimento DATE,
    raca            VARCHAR(50),
    peso            DECIMAL(6,2),
    status_saude    VARCHAR(50)
);

CREATE TABLE Veterinario (
    id                    SERIAL PRIMARY KEY,
    nome_completo         VARCHAR(100),
    telefone              VARCHAR(20),
    registro_profissional VARCHAR(50)
);

CREATE TABLE Funcionario (
    id            SERIAL PRIMARY KEY,
    nome_completo VARCHAR(100),
    telefone      VARCHAR(20),
    turno_periodo VARCHAR(50)
);

CREATE TABLE Supervisor (
    id               SERIAL PRIMARY KEY,
    nome_completo    VARCHAR(100),
    telefone         VARCHAR(20),
    turno_supervisao VARCHAR(50)
);
```

Listing 2: Proprietário, Equipamentos, Vaca, Veterinário, Funcionário, Supervisor

3.2 Tabela fato: Ordenha

A tabela `Ordenha` representa o **evento central** do sistema. Ela depende de `Proprietario` (1:N) e `Vaca` (1:N). As FKs são adicionadas separadamente com `ALTER TABLE` para maior clareza.

```sql
CREATE TABLE Ordenha (
    id              SERIAL PRIMARY KEY,
    data            DATE,
```

```
    horario          TIME,
    quantidade_leite DECIMAL(6,2),
    id_proprietario  INT,           -- FK -> Proprietario
    id_vaca          INT NOT NULL   -- FK -> Vaca (obrigatorio)
);

-- Chave estrangeira: Ordenha -> Proprietario (1:N)
ALTER TABLE Ordenha
ADD CONSTRAINT fk_ordenha_proprietario
FOREIGN KEY (id_proprietario) REFERENCES Proprietario(id);

-- Chave estrangeira: Ordenha -> Vaca (1:N)
ALTER TABLE Ordenha
ADD CONSTRAINT fk_ordenha_vaca
FOREIGN KEY (id_vaca) REFERENCES Vaca(id);
```

Listing 3: Criação da tabela Ordenha e suas FKs 1:N

Dica Prática

Observe que `id_vaca` é declarado `NOT NULL`: toda ordenha obrigatoriamente pertence a uma vaca. Já `id_proprietario` pode ser nulo — um evento pode ser registrado antes de vincular um proprietário.

4 Passo 3 — Tabelas Associativas (N:N)

Relacionamentos N:N são implementados com uma **tabela intermediária** cuja chave primária é composta pelas duas FKs envolvidas.

```
-- Vaca <-> Veterinario
CREATE TABLE Vaca_Veterinario (
    id_veterinario INT NOT NULL,
    id_vaca        INT NOT NULL,
    PRIMARY KEY (id_veterinario, id_vaca),
    FOREIGN KEY (id_veterinario) REFERENCES Veterinario(id),
    FOREIGN KEY (id_vaca)        REFERENCES Vaca(id)
);

-- Ordenha <-> Funcionario
CREATE TABLE Ordenha_Funcionario (
    id_ordenha    INT NOT NULL,
    id_funcionario INT NOT NULL,
    PRIMARY KEY (id_ordenha, id_funcionario),
    FOREIGN KEY (id_ordenha)     REFERENCES Ordenha(id),
    FOREIGN KEY (id_funcionario) REFERENCES Funcionario(id)
);
```

```sql

-- Ordenha <-> Supervisor
CREATE TABLE Ordenha_Supervisor (
    id_ordenha   INT NOT NULL,
    id_supervisor INT NOT NULL,
    PRIMARY KEY (id_ordenha, id_supervisor),
    FOREIGN KEY (id_ordenha)    REFERENCES Ordenha(id),
    FOREIGN KEY (id_supervisor) REFERENCES Supervisor(id)
);

-- Ordenha <-> Equipamentos
CREATE TABLE Ordenha_Equipamento (
    id_ordenha         INT NOT NULL,
    codigo_equipamento INT NOT NULL,
    PRIMARY KEY (id_ordenha, codigo_equipamento),
    FOREIGN KEY (id_ordenha)         REFERENCES Ordenha(id),
    FOREIGN KEY (codigo_equipamento) REFERENCES
       Equipamentos(codigo_equipamento)
);

-- Proprietario <-> Equipamentos
CREATE TABLE Proprietario_Equipamento (
    id_proprietario    INT NOT NULL,
    codigo_equipamento INT NOT NULL,
    PRIMARY KEY (id_proprietario, codigo_equipamento),
    FOREIGN KEY (id_proprietario)    REFERENCES Proprietario(id),
    FOREIGN KEY (codigo_equipamento) REFERENCES
       Equipamentos(codigo_equipamento)
);

-- Funcionario <-> Supervisor
CREATE TABLE Funcionario_Supervisor (
    id_funcionario INT NOT NULL,
    id_supervisor  INT NOT NULL,
    PRIMARY KEY (id_funcionario, id_supervisor),
    FOREIGN KEY (id_funcionario) REFERENCES Funcionario(id),
    FOREIGN KEY (id_supervisor)  REFERENCES Supervisor(id)
);
```

Listing 4: Todas as tabelas associativas (N:N)

Resumo das tabelas associativas criadas:

Tabela Associativa	Entidade A	Entidade B
Vaca_Veterinario	Vaca	Veterinário
Ordenha_Funcionario	Ordenha	Funcionário
Ordenha_Supervisor	Ordenha	Supervisor
Ordenha_Equipamento	Ordenha	Equipamentos
Proprietario_Equipamento	Proprietário	Equipamentos
Funcionario_Supervisor	Funcionário	Supervisor

5 Passo 4 — Inserir Dados (DML)

5.1 Inserção nas tabelas principais

```sql
-- Proprietarios
INSERT INTO Proprietario (nome_completo, telefone) VALUES
('Joao Silva',     '62999990001'),
('Maria Souza',    '62999990002'),
('Carlos Pereira','62999990003');

-- Equipamentos
INSERT INTO Equipamentos (tipo, ultima_manutencao, situacao) VALUES
('Ordenhadeira', '2026-03-01', 'Ativo'),
('Resfriador',   '2026-02-15', 'Ativo'),
('Bomba de Leite','2026-01-10', 'Manutencao');

-- Vacas
INSERT INTO Vaca (data_nascimento, raca, peso, status_saude) VALUES
('2020-05-10', 'Holandesa', 550.00, 'Saudavel'),
('2019-08-20', 'Jersey',    480.00, 'Saudavel'),
('2021-02-15', 'Gir',       520.00, 'Tratamento');

-- Veterinarios
INSERT INTO Veterinario (nome_completo, telefone,
   registro_profissional) VALUES
('Dr. Carlos Lima', '62988880001', 'CRMV123'),
('Dra. Ana Paula',  '62988880002', 'CRMV456');

-- Funcionarios
INSERT INTO Funcionario (nome_completo, telefone, turno_periodo)
   VALUES
('Pedro Santos',  '62977770001', 'Manha'),
('Lucas Oliveira','62977770002', 'Tarde'),
('Bruno Alves',   '62977770003', 'Noite');

-- Supervisores
INSERT INTO Supervisor (nome_completo, telefone, turno_supervisao)
   VALUES
```

```
('Marcos Silva',   '62966660001', 'Manha'),
('Fernanda Costa', '62966660002', 'Noite');
```

Listing 5: INSERT nas entidades independentes

5.2 Inserção na tabela Ordenha (1:N)

```
-- Relacionamento Ordenha -> Proprietario (id) e Ordenha -> Vaca (id)
INSERT INTO Ordenha (data, horario, quantidade_leite, id_proprietario, id_vaca) VALUES
('2026-04-10', '06:00', 25.5, 1, 1),   -- Joao Silva / Vaca Holandesa
('2026-04-10', '18:00', 27.0, 1, 2),   -- Joao Silva / Vaca Jersey
('2026-04-11', '06:00', 26.3, 2, 1),   -- Maria Souza / Vaca Holandesa
('2026-04-11', '18:00', 24.8, 3, 3);   -- Carlos Pereira / Vaca Gir
```

Listing 6: INSERT na tabela fato Ordenha

5.3 Inserção nas tabelas associativas (N:N)

```
-- Vaca <-> Veterinario
INSERT INTO Vaca_Veterinario (id_veterinario, id_vaca) VALUES
(1, 1),  -- Dr. Carlos Lima atende Vaca 1 (Holandesa)
(1, 2),  -- Dr. Carlos Lima atende Vaca 2 (Jersey)
(2, 3),  -- Dra. Ana Paula atende Vaca 3 (Gir)
(2, 1);  -- Dra. Ana Paula atende Vaca 1 (Holandesa)

-- Ordenha <-> Funcionario
INSERT INTO Ordenha_Funcionario (id_ordenha, id_funcionario) VALUES
(1, 1),  -- Ordenha 1 com Pedro Santos
(1, 2),  -- Ordenha 1 com Lucas Oliveira
(2, 1),  -- Ordenha 2 com Pedro Santos
(3, 2),  -- Ordenha 3 com Lucas Oliveira
(4, 3);  -- Ordenha 4 com Bruno Alves

-- Ordenha <-> Supervisor
INSERT INTO Ordenha_Supervisor (id_ordenha, id_supervisor) VALUES
(1, 1),  -- Ordenha 1 supervisionada por Marcos Silva
(2, 1),  -- Ordenha 2 supervisionada por Marcos Silva
(3, 2),  -- Ordenha 3 supervisionada por Fernanda Costa
(4, 2);  -- Ordenha 4 supervisionada por Fernanda Costa

-- Ordenha <-> Equipamentos
```

```sql
INSERT INTO Ordenha_Equipamento (id_ordenha, codigo_equipamento)
    VALUES
(1, 1),   -- Ordenha 1 usou Ordenhadeira
(1, 2),   -- Ordenha 1 usou Resfriador
(2, 1),   -- Ordenha 2 usou Ordenhadeira
(3, 2),   -- Ordenha 3 usou Resfriador
(4, 3);   -- Ordenha 4 usou Bomba de Leite

-- Proprietario <-> Equipamentos
INSERT INTO Proprietario_Equipamento (id_proprietario,
    codigo_equipamento) VALUES
(1, 1),   -- Joao Silva possui Ordenhadeira
(1, 2),   -- Joao Silva possui Resfriador
(2, 2),   -- Maria Souza possui Resfriador
(3, 3);   -- Carlos Pereira possui Bomba de Leite

-- Funcionario <-> Supervisor
INSERT INTO Funcionario_Supervisor (id_funcionario, id_supervisor)
    VALUES
(1, 1),   -- Pedro Santos supervisionado por Marcos Silva
(2, 1),   -- Lucas Oliveira supervisionado por Marcos Silva
(2, 2),   -- Lucas Oliveira supervisionado por Fernanda Costa
(3, 2);   -- Bruno Alves supervisionado por Fernanda Costa
```

Listing 7: INSERT nas tabelas N:N

6 Passo 5 — Consultas de Verificação (SELECT)

Após inserir os dados, execute as consultas abaixo para verificar a integridade.

```sql
-- (a) Todas as ordenhas com nome do proprietario e raca da vaca
SELECT
    o.id           AS ordenha_id,
    o.data,
    o.horario,
    o.quantidade_leite,
    p.nome_completo AS proprietario,
    v.raca
FROM Ordenha o
JOIN Proprietario p ON o.id_proprietario = p.id
JOIN Vaca         v ON o.id_vaca         = v.id
ORDER BY o.data, o.horario;

-- (b) Funcionarios por ordenha
SELECT
    o.id   AS ordenha_id,
```

```
    o.data,
    f.nome_completo AS funcionario,
    f.turno_periodo
FROM Ordenha_Funcionario of_
JOIN Ordenha    o ON of_.id_ordenha    = o.id
JOIN Funcionario f ON of_.id_funcionario = f.id
ORDER BY o.id;

-- (c) Quais vacas cada veterinario atende
SELECT
    vet.nome_completo AS veterinario,
    v.raca,
    v.status_saude
FROM Vaca_Veterinario vv
JOIN Veterinario vet ON vv.id_veterinario = vet.id
JOIN Vaca        v   ON vv.id_vaca        = v.id
ORDER BY vet.nome_completo;

-- (d) Total de leite produzido por proprietario
SELECT
    p.nome_completo AS proprietario,
    SUM(o.quantidade_leite) AS total_litros
FROM Ordenha o
JOIN Proprietario p ON o.id_proprietario = p.id
GROUP BY p.nome_completo
ORDER BY total_litros DESC;

-- (e) Equipamentos utilizados em cada ordenha
SELECT
    o.id     AS ordenha_id,
    o.data,
    e.tipo   AS equipamento,
    e.situacao
FROM Ordenha_Equipamento oe
JOIN Ordenha     o ON oe.id_ordenha         = o.id
JOIN Equipamentos e ON oe.codigo_equipamento = e.codigo_equipamento
ORDER BY o.id;
```

Listing 8: Consultas básicas de verificação

7 Exercícios Práticos

Instruções

Para cada exercício: execute o SQL no terminal do PostgreSQL (`psql`), observe o resultado e responda a pergunta correspondente. Registre o resultado na caixa abaixo de cada item.

Ex. 1: **(Integridade Referencial)** Tente inserir uma ordenha para uma vaca com `id = 99` (que não existe). O banco aceita? Por quê?

```sql
INSERT INTO Ordenha (data, horario, quantidade_leite,
   id_proprietario, id_vaca)
VALUES ('2026-04-12', '06:00', 20.0, 1, 99);
```

Listing 9: Teste de integridade

O que aconteceu? Qual mensagem de erro apareceu?

Resposta:

Ex. 2: **(SELECT com JOIN)** Escreva uma consulta que liste o nome do supervisor e todos os funcionários sob sua supervisão, mostrando o turno de cada funcionário.

```sql
SELECT
    s.nome_completo  AS supervisor,
    -- complete aqui
FROM Funcionario_Supervisor fs
JOIN Supervisor   s ON fs.id_supervisor  = s.id
JOIN Funcionario  f ON fs.id_funcionario = f.id
ORDER BY s.nome_completo;
```

Listing 10: Modelo inicial — complete o SELECT

Resposta:

Ex. 3: **(Agregação)** Quantas ordenhas foram realizadas por cada supervisor? Escreva a consulta usando `COUNT` e `GROUP BY`.

Resposta:

Ex. 4: **(Atualização)** O equipamento "Bomba de Leite" (código 3) concluiu a manutenção. Atualize seu status para `'Ativo'` e a data de última manutenção para hoje.

```
UPDATE Equipamentos
SET situacao = _____, ultima_manutencao = _____
WHERE codigo_equipamento = _____;
```

Listing 11: Modelo UPDATE — complete

Resposta:

Ex. 5: **(Análise de Dados)** Qual vaca produziu o maior volume total de leite nas quatro ordenhas registradas? Escreva a consulta usando `SUM`, `GROUP BY` e `ORDER BY`.

Resposta:

Ex. 6: **(Reflexão — Modelo)** O modelo possui a tabela `Funcionario_Supervisor` (N:N). Em que situação real isso seria necessário? Poderia existir um relacionamento 1:N entre Funcionário e Supervisor? Justifique com base nos dados inseridos.

Resposta:

Ex. 7: **(Extensão do Modelo)** Suponha que se queira registrar o **resultado da consulta veterinária** (data, diagnóstico e medicamento) para cada par Vaca–Veterinário. Como ficaria a nova estrutura da tabela `Vaca_Veterinario`? Escreva o `ALTER TABLE` ou o novo `CREATE TABLE` correspondente.

Resposta:

Checklist de Entrega

- ☐ Script DDL completo executado sem erros.
- ☐ Script DML (INSERTs) executado com sucesso.
- ☐ Todas as 5 consultas de verificação retornaram resultados corretos.
- ☐ Exercícios 1 a 7 respondidos com SQL e justificativa.
- ☐ Arquivo `.sql` com o script completo entregue no Classroom.

"Um banco de dados bem modelado é como uma fazenda bem organizada: cada coisa em seu lugar, cada lugar com seu dono."